

Grupo 7 - Examen Práctico RL

Integrantes:

- Diego Valenzuela 22309
- Daniel Dubón 22233
- Joaquín Puente 22296
- Christian Echeverria 221441
- Mario Betancurt 23440

Repo: <https://github.com/DanielDubon/RL-EX1>

```
In [1]: import numpy as np
from collections import defaultdict

# ESPECIFICACIÓN DEL MDP
# Estado: (nivel_inventario, dias_hasta_vencimiento, demanda_promedio_7dias)
# nivel_inventario: [0, 10, 20, ..., 100] - 10 niveles
# dias_hasta_vencimiento: [1, 7, 14, 30, 60] - 5 niveles
# demanda_promedio_7dias: [bajo, medio, alto, crítico] - 4 niveles
# Total: 200 estados | Acciones: [0, 10, 20, 30, 40, 50] unidades - 6 acciones

def transition(state, action):
    inventory, days_to_expiry, demand_level = state
    demand_map = {'bajo': 5, 'medio': 15, 'alto': 25, 'crítico': 40}
    daily_demand = demand_map[demand_level]
    new_inventory = min(100, max(0, inventory + action - daily_demand))
    new_days = max(1, days_to_expiry - 1)
    new_demand = demand_level
    return (new_inventory, new_days, new_demand)

def reward(state, action, next_state):
    new_inventory, new_days, _ = next_state
    inventory_reward = new_inventory * 0.5
    expiry_penalty = -10 if new_days <= 7 else 0
    order_penalty = -2 if action > 0 else 0
    return inventory_reward + expiry_penalty + order_penalty

def train(env, episodes=1000):
    Q = defaultdict(lambda: np.zeros(6))
    alpha = 0.9
    gamma = 0.99
    epsilon = 0.05
    for episode in range(episodes):
        state = env.reset()
        done = False
        while not done:
            if np.random.random() < epsilon:
                action = np.random.randint(6)
            else:
```

```

        action = np.argmax(Q[state])
        next_state, reward, done, _ = env.step(action)
        next_action = np.argmax(Q[next_state])
        td_target = reward + gamma * Q[next_state][next_action]
        Q[state][action] += alpha * (td_target - Q[state][action])
        state = next_state
    return Q

def preprocess_state(state):
    inventory, days_to_expiry, demand_level = state
    demand_map = {'bajo': 0, 'medio': 1, 'alto': 2, 'crítico': 3}
    features = np.array([
        inventory,
        days_to_expiry,
        demand_map[demand_level]
    ])
    return features

class LinearApproximator:
    def __init__(self, n_features=3, n_actions=6):
        self.weights = np.zeros((n_actions, n_features))
    def predict(self, features, action):
        return np.dot(self.weights[action], features)
    def update(self, features, action, target, alpha=0.01):
        prediction = self.predict(features, action)
        error = target - prediction
        self.weights[action] += alpha * error * features

def evaluate_policy(Q, env, episodes=100):
    total_rewards = []
    for episode in range(episodes):
        state = env.reset()
        episode_reward = 0
        done = False
        while not done:
            action = np.argmax(Q[state])
            next_state, reward, done, _ = env.step(action)
            episode_reward += reward
            state = next_state
        total_rewards.append(episode_reward)
    return {
        'mean_reward': np.mean(total_rewards),
        'std_reward': np.std(total_rewards),
        'min_reward': np.min(total_rewards)
    }

# MÉTRICAS DE ENTRENAMIENTO
# Episodio    Recompensa    Error TD    Política dominante
# 100         42.3         8.42       pedir_50
# 500         48.1         7.12       pedir_50
# 1000        51.7         5.21       pedir_50
# Varianza entre episodios: 0.8
# Política greedy: pedir 50 en 94% de estados

# RESULTADOS EN PRODUCCIÓN
resultados_produccion = {

```

```

    'stockouts_por_semana': 23,
    'productos_vencidos_por_semana': 41,
    'costo_almacenamiento_semanal': 8400,
    'costo_objetivo_semanal': 3200,
    'satisfaccion_cliente': 0.61
}

# RESULTADOS EN SIMULACIÓN
resultados_simulacion = {
    'mean_reward': 51.2,
    'std_reward': 0.9,
    'min_reward': 48.3
}

```

Explicación matemática de las fórmulas del código

Para describir las operaciones se usa el estado $s_t = (I_t, D_t, L_t)$, donde I_t es el inventario, D_t son los días restantes hasta el vencimiento y L_t es el nivel de demanda. La acción a_t representa las unidades solicitadas y $d(L_t)$ transforma el nivel de demanda en unidades mediante el mapa `bajo=5`, `medio=15`, `alto=25` y `crítico=40`.

1. Transición del inventario

$$I_{t+1} = \min(100, \max(0, I_t + a_t - d(L_t)))$$

Primero se suma el pedido al inventario y se resta la demanda. El máximo con 0 impide inventarios negativos y el mínimo con 100 aplica la capacidad máxima. Este recorte también oculta cuántas unidades de demanda quedaron sin atender: cualquier resultado negativo se convierte simplemente en cero.

Los días restantes se actualizan con

$$D_{t+1} = \max(1, D_t - 1).$$

El contador disminuye un día por paso, pero nunca baja de 1. Además, $L_{t+1} = L_t$, por lo que la categoría de demanda queda fija durante el episodio.

2. Función de recompensa

La función `reward` puede escribirse como

$$r_t = 0.5I_{t+1} - 10 \mathbf{1}[D_{t+1} \leq 7] - 2 \mathbf{1}[a_t > 0],$$

donde $\mathbf{1}[C]$ vale 1 si la condición C se cumple y 0 en caso contrario. El primer término premia cada unidad almacenada con 0.5 puntos; el segundo descuenta 10 puntos si quedan siete días o menos; el tercero descuenta 2 puntos por hacer cualquier pedido, sin importar su tamaño. Como $0 \leq I_{t+1} \leq 100$, el premio de inventario está entre 0 y 50, mientras que las dos penalizaciones suman como mínimo -12. Esto explica por qué la recompensa favorece inventarios altos.

3. Selección epsilon-greedy

Con $\epsilon = 0.05$, el agente explora una acción aleatoria con probabilidad 0.05 y elige una acción greedy con probabilidad 0.95:

$$a_t = \begin{cases} \text{acción aleatoria,} & \text{con probabilidad } \epsilon, \\ \arg \max_a Q(s_t, a), & \text{con probabilidad } 1 - \epsilon. \end{cases}$$

Como la acción aleatoria también puede coincidir con la greedy, con seis acciones la probabilidad total de ejecutar la acción greedy es $0.95 + 0.05/6 \approx 0.9583$.

4. Actualización Q-learning

El objetivo temporal y el error TD son

$$y_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a'), \quad \delta_t = y_t - Q(s_t, a_t).$$

Después se actualiza únicamente el valor de la pareja observada:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \delta_t.$$

En el código, $\alpha = 0.9$ controla cuánto pesa la nueva información y $\gamma = 0.99$ cuánto se valoran las recompensas futuras. `next_action = argmax(Q[next_state])` implementa el máximo de Q-learning; no significa que esa acción deba ejecutarse en el siguiente paso. El código conserva el término futuro incluso si el paso es terminal.

5. Aproximación lineal

`preprocess_state` construye el vector $x(s) = [I, D, m(L)]^T$, donde $m(L) \in \{0, 1, 2, 3\}$. Para cada acción, el valor estimado es el producto punto

$$\hat{Q}(s, a) = w_a^T x(s).$$

Si el objetivo es y , el error es $e = y - \hat{Q}(s, a)$ y los pesos se corrigen mediante $w_a \leftarrow w_a + \alpha e x(s)$. Cada característica modifica los pesos en proporción a su valor y al error. Esta clase está definida, pero `train` usa la tabla `Q` y no llama al aproximador.

6. Métricas de evaluación

Para cada episodio i , el retorno es $G_i = \sum_t r_{i,t}$. Con N episodios, `evaluate_policy` calcula

$$\bar{G} = \frac{1}{N} \sum_{i=1}^N G_i, \quad \sigma_G = \sqrt{\frac{1}{N} \sum_{i=1}^N (G_i - \bar{G})^2}, \quad G_{\min} = \min_i G_i.$$

Estas expresiones corresponden a `np.mean`, `np.std` con su configuración poblacional por defecto y `np.min`. Solo resumen la recompensa acumulada; no calculan directamente stockouts, vencimientos ni costos reales.

Entregable 7.1 — Validación del gap simulación-producción

Se valida la dirección del análisis previo, pero se precisan las atribuciones y se separa lo demostrado por el código de lo que solo es una hipótesis. El cociente real de costo es $8400/3200 = 2.625$, es decir, aproximadamente 2.6x el objetivo.

Síntoma observado	Causa probable y componente	Evidencia en el código
Costo de almacenamiento 2.6x y política pedir_50 en 94%	Función de recompensa mal alineada. Premia mantener inventario y casi no diferencia el tamaño del pedido.	<code>inventory_reward = new_inventory * 0.5</code> entrega hasta +50 por paso; <code>order_penalty = -2 if action > 0 else 0</code> cobra lo mismo por 10 que por 50 unidades. Sobreabastecer puede maximizar el retorno simulado aunque sea caro en producción.
41 productos vencidos/semana	Reward y modelo de transición incompletos. El costo de vencer es binario y no proporcional a unidades; el estado tampoco representa lotes con edades distintas.	<code>expiry_penalty = -10 if new_days <= 7 else 0</code> aplica la misma penalización con una o cien unidades. <code>new_days = max(1, days_to_expiry - 1)</code> solo baja un contador y nunca descarta inventario vencido ni reinicia la vida útil de las unidades nuevas. El simulador no reproduce las 41 mermas.
23 stockouts/semana	Reward sin costo de demanda perdida, transición que oculta el faltante, y tabla Q sin generalización.	<code>max(0, inventory + action - daily_demand)</code> recorta el inventario en cero, pero no conserva <code>lost_sales</code> ni penaliza su magnitud. <code>defaultdict(lambda: np.zeros(6))</code> deja sin conocimiento los estados no vistos; la política greedy elige allí la acción 0.
Satisfacción 0.61	Objetivo de entrenamiento/evaluación distinto al KPI real. Servicio, stockouts y satisfacción no aparecen en el retorno.	<code>reward(...)</code> solo incluye inventario, umbral de vencimiento y un costo fijo por ordenar. <code>evaluate_policy(...)</code> reporta exclusivamente el retorno de ese mismo reward.
51.2 estable en simulación pero mal resultado real	Evaluación sin cambio de distribución. Se evalúa con el mismo <code>env</code> y las mismas simplificaciones usadas al entrenar.	<code>evaluate_policy(Q, env)</code> llama <code>env.reset()</code> / <code>env.step()</code> igual que <code>train</code> . La baja desviación (0.9) mide consistencia dentro de ese simulador, no robustez ante demanda o estados de producción. <code>new_demand = demand_level</code> además impide cambios de demanda dentro de cada episodio.

Precisión importante: `epsilon = 0.05` solo explora acciones. No puede corregir por sí sola una transición que mantiene fija la demanda ni una distribución de `reset()` que omite estados. Aunque existe `LinearApproximator`, `train` y `evaluate_policy` usan exclusivamente la tabla `Q`, así que no hay generalización a tuplas nuevas.

Entregable 7.2 — Política greedy en estados no visitados

```
In [2]: # Auditoría reproducible de la estimación y de la política greedy.
TOTAL_ESTADOS_DECLARADOS = 200
PORCENTAJE_PEDIR_50 = 0.94
ACCIONES_EN_UNIDADES = [0, 10, 20, 30, 40, 50]

# Lectura prevista del dato agregado: si el 94% se calculó sobre los 200,
# 6% (= 12 estados) no elige pedir 50. Todo estado no visitado cae en este
# grupo porque conserva seis ceros y argmax desempata hacia el índice 0.
estados_que_no_eligen_50 = round(
    TOTAL_ESTADOS_DECLARADOS * (1 - PORCENTAJE_PEDIR_50)
)

q_estado_no_visitado = np.zeros(6)
accion_idx = int(np.argmax(q_estado_no_visitado))
accion_unidades = ACCIONES_EN_UNIDADES[accion_idx]
siguiente_estado_critico = transition((0, 1, 'crítico'), accion_unidades)

print(f"Estados fuera del 94%: {estados_que_no_eligen_50}/200")
print("Estimación probable de no visitados: ≈12/200 (6%), no un conteo exacto")
print(f"Q(s) no visitado: {q_estado_no_visitado}")
print(f"argmax -> índice {accion_idx} -> pedir {accion_unidades} unidades")
print(f"Desde (0, 1, 'crítico') se pasa a {siguiente_estado_critico}")
```

```
Estados fuera del 94%: 12/200
Estimación probable de no visitados: ≈12/200 (6%), no un conteo exacto
Q(s) no visitado: [0. 0. 0. 0. 0. 0.]
argmax -> índice 0 -> pedir 0 unidades
Desde (0, 1, 'crítico') se pasa a (0, 1, 'crítico')
```

Explicación matemática de la estimación

El código calcula los estados que no pertenecen al 94% asociado con `pedir_50` mediante

$$N_{\text{fuera}} = \text{round}(200(1 - 0.94)) = \text{round}(12) = 12.$$

El complemento $1 - 0.94 = 0.06$ representa el 6% restante. Esta operación demuestra que 12 estados no eligen `pedir_50`, pero no demuestra que los 12 nunca se visitaron. Por eso el notebook los presenta como una estimación y, con la información disponible, como una cota máxima de estados no visitados.

Para un estado no visitado, $Q(s) = [0, 0, 0, 0, 0, 0]$. Todas las acciones empatan, y `np.argmax` devuelve el primer índice del máximo. Por tanto, $\arg \max_a Q(s, a) = 0$, que en `ACCIONES_EN_UNIDADES` corresponde a pedir 0 unidades.

1. ¿Cuántos estados probablemente no se visitaron?

La estimación más directa que permite el dato agregado es aproximadamente 12 de 200 estados (6%):

- La política reportada pide 50 en 94% de los 200 estados: $0.94(200) = 188$.
- Quedan $200 - 188 = 12$ estados donde la acción greedy no es 50.
- Un estado no visitado conserva `Q[s] = [0, 0, 0, 0, 0, 0]` y por tanto elige la acción 0; necesariamente queda fuera del 94% que elige 50.

Esta cifra es una estimación, no una igualdad demostrable. Formalmente, si el 94% fue calculado sobre los 200 estados, el código permite concluir que hay como máximo 12 no visitados; algunos de esos 12 también podrían ser estados visitados que aprendieron legítimamente otra acción. La aproximación "12 no visitados" supone que las excepciones a la política dominante se deben principalmente a valores Q que nunca se actualizaron.

El riesgo de cobertura es plausible porque `new_demand = demand_level` impide cambiar de categoría de demanda dentro de un episodio, y `epsilon = 0.05` solo explora acciones, no categorías de estado. Sin embargo, el archivo no incluye `env.reset()`, el horizonte, la tabla Q final ni un conjunto `visited_states`; por eso no es válido fabricar un conteo exacto a partir de `len(Q)`. Q también puede contener estados consultados para bootstrap o evaluación, y la transición genera coordenadas fuera de la grilla, como días 29, 28, etc.

2. ¿Qué hace la política greedy en un estado no visitado?

`Q = defaultdict(lambda: np.zeros(6))` crea seis ceros al consultar una tupla desconocida. Tanto `train` como `evaluate_policy` usan `np.argmax(Q[state])`. Cuando todos los valores empatan, NumPy devuelve la primera posición: índice 0. Con el orden declarado `[0, 10, 20, 30, 40, 50]`, el agente decide pedir 0 unidades. Crear la entrada al consultarla no significa que el estado haya sido aprendido.

3. ¿Cómo genera stockouts?

En un estado de inventario bajo y demanda alta o crítica, ordenar cero hace que la demanda supere a `inventory + action`. La transición oculta el faltante al recortar con `max(0, ...)`: desde `(0, 1, 'crítico')`, cuya demanda diaria es 40, la acción 0 devuelve de nuevo inventario 0. Como el reward no registra unidades de demanda insatisfecha y el código de producción no muestra aprendizaje en línea, la política puede repetir "no pedir" y prolongar el quiebre. Este mecanismo es consistente con los 23 stockouts observados, aunque el código no permite afirmar que explique exactamente los 23.

Controles de consistencia

1. El 94% debe tener como denominador los 200 estados para sostener la estimación de 12. Si fue calculado solo sobre claves visitadas/presentes en `Q`, no informa cuántos estados faltan y el conteo exacto queda indeterminado.
2. La especificación tiene una contradicción adicional: `[0, 10, ..., 100]` contiene 11 valores, no 10; la grilla literal tendría $11 \times 5 \times 4 = 220$ estados. Aquí se usa 200 porque es el total exigido por el enunciado.
3. Para obtener el número real habría que registrar `visited_states.add(state)` dentro de `train` y comparar ese conjunto con la grilla válida; consultar después un `defaultdict` no es una medición confiable.

Conclusión: bajo la lectura prevista de la métrica del 94%, unos 12 de 200 estados probablemente no se visitaron. En cualquiera de ellos la política greedy elige no pedir, lo que convierte una falta de cobertura del entrenamiento en stockout cuando el estado desconocido tiene poco inventario y demanda elevada.

Actualización tras recibir el insumo del Grupo 3 (sesión presencial)

Nuestra estimación de " $\sim 12/200$ no visitados" arriba asumía que el denominador correcto era 200 estados (el número declarado en el enunciado). El Grupo 3 replicó el entorno original 200 veces (200 semillas x 1000 episodios) y midió directamente 4,371 estados alcanzables. Confirma por otra vía lo que ya habíamos señalado en 7.1: `transition()` no discretiza el inventario a la grilla `[0, 10, ..., 100]`, así que el espacio real de estados es mucho mayor al nominal.

Con ese denominador, la cobertura final de pares `(s, a)` que reporta G3 es $1370.8 / 26226 (\approx 5.2\%)$ para el algoritmo original, no $\sim 94\%$ de 200 estados aprendidos sino una fracción pequeña de un espacio mucho más grande. Esto refuerza, no contradice, la conclusión cualitativa de 7.2: la política greedy elige "pedir 0" por defecto en estados no vistos y eso genera stockouts. Pero el problema de cobertura es órdenes de magnitud más grave de lo que el análisis inicial, basado solo en el código y el 94% reportado, permitía estimar. Ajustamos esta cifra en la Pregunta 2 con el detalle completo del bug identificado por G3.

Entregable 7.3 — Protocolo de evaluación mejorado

El protocolo actual (`evaluate_policy`) falla porque reutiliza el mismo `env` y la misma distribución de estados que `train`: mide consistencia interna del simulador, no transferencia a producción. Proponemos un protocolo de tres capas que se ejecuta antes de cualquier despliegue.

Capa 1, métricas de cobertura de entrenamiento (gate previo a evaluar):

- `estados_visitados / 200` y `pares_(s,a)_visitados / 1200`.

- Condición de aceptación: al menos 95% de cobertura de estados y 90% de cobertura de pares `(s,a)`, con cobertura mínima del 80% dentro de cada categoría de demanda (incluyendo `crítico`), no solo en promedio global.
- Si no se cumple, el entrenamiento se rechaza antes de tocar métricas de recompensa. Cobertura insuficiente invalida cualquier resultado posterior (ver Entregable 7.2).

Capa 2, evaluación fuera de distribución (estrés dirigido):

- Conjunto de prueba separado que sobre-muestrea deliberadamente los estados de baja frecuencia: inventario bajo (0-20) combinado con demanda `alto / crítico`, y `dias_hasta_vencimiento` bajo (1-7).
- Métrica: tasa de stockout simulado (`next_inventory == 0` bajo demanda no satisfecha) y tasa de vencimiento simulado, medidas por separado del reward agregado. Un reward promedio saludable puede ocultar colas malas.
- Condición de aceptación: la tasa de stockout simulado en el subconjunto de estrés no debe superar en más de 2x la tasa observada en el conjunto de prueba general.

Capa 3, réplica de KPIs de negocio, no solo de reward:

- Traducir el reward a las mismas unidades que reporta producción: stockouts/semana, vencidos/semana, costo de almacenamiento/semana, satisfacción. `evaluate_policy` actual no calcula ninguno de estos directamente.
- Condición de aceptación: proyección de costo de almacenamiento simulado dentro de $\pm 20\%$ del costo objetivo (3200), antes de autorizar el paso a producción.
- Piloto controlado: desplegar la política nueva en un subconjunto pequeño de tiendas/productos durante 1-2 semanas y comparar KPIs reales contra la proyección de la Capa 3 antes del rollout completo.

Este protocolo detecta el gap actual (51.2 en simulación vs. 23 stockouts reales) porque la Capa 1 habría bloqueado el despliegue por cobertura insuficiente, y la Capa 3 habría expuesto que el reward optimizado no corresponde a los KPIs reales de negocio.

Implementación (código de nuestro componente): `protocolo_evaluacion.py` implementa las tres capas como funciones (`gate_cobertura`, `prueba_estres`, `proyeccion_kpis`) orquestadas por `protocolo_evaluacion()`, que corta la evaluación si la Capa 1 rechaza. La celda siguiente lo corre sobre las dos políticas del Grupo 5 (epsilon-greedy original y Optimista+UCB) para mostrar que el protocolo distingue entre una política que no debería desplegarse y una que sí:

```
In [3]: # Implementacion del protocolo de 3 capas descrito arriba (protocolo_evaluacion.py)
# Corre sobre el entorno y las politicas del Grupo 5 (exploracion_grupo5.py)
# que ya entrega Q y sa_pairs_visited listos para auditar.
#
# Relacion con formulas de las diapositivas: gate_cobertura verifica la
# condicion de exploracion suficiente que garantiza que la politica greedy
```

```

#  $\pi(s) = \operatorname{argmax}_a Q(s,a)$  fue calculada sobre estimaciones  $Q(s,a)$  con al
# menos una actualizacion  $Q(s,a) \leftarrow Q(s,a) + \alpha \cdot (TD\_target - Q(s,a))$ ;
# un par  $(s,a)$  nunca visitado deja  $Q(s,a)$  en su valor de inicializacion,
# por lo que  $\operatorname{argmax}_a Q(s,a)$  no refleja ninguna experiencia real.

from protocolo_evaluacion import (
    protocolo_evaluacion, PharmacyInventoryEnv,
    train_epsilon_greedy, train_optimistic_ucb,
)

env_protocolo = PharmacyInventoryEnv()

np.random.seed(42)
Q_original_demo, _vc_orig, sap_original_demo = train_epsilon_greedy(env_prot

np.random.seed(42)
Q_propuesta_demo, _vc_prop, sap_propuesta_demo = train_optimistic_ucb(env_pr

_ = protocolo_evaluacion(env_protocolo, Q_original_demo, sap_original_demo,
_ = protocolo_evaluacion(env_protocolo, Q_propuesta_demo, sap_propuesta_demo

```

=== Protocolo de evaluacion – epsilon-greedy (original) ===

Capa 1 (cobertura): RECHAZADO – pares 64.2%, estados 100.0%, por demanda {'bajo': 70.0, 'medio': 72.7, 'alto': 64.0, 'crítico': 50.0}

-> Despliegue bloqueado en Capa 1. No se evalua Capa 2 ni 3 (cobertura insuficiente invalida cualquier metrica posterior).

=== Protocolo de evaluacion – Optimista+UCB (Grupo 5) ===

Capa 1 (cobertura): APROBADO – pares 99.6%, estados 100.0%, por demanda {'bajo': 100.0, 'medio': 100.0, 'alto': 100.0, 'crítico': 98.3}

Capa 2 (estres): APROBADO – stockout general 0.0%, stockout en estres 0.0% (40 estados de estres)

Capa 3 (KPIs): stockouts/semana proyectados = 0.0, vencimientos/semana = 2.31, inventario promedio = 80.69

Nota: sin conversion a \$/semana: falta costo por unidad almacenada en el código del examen

>>> Resultado final: APROBADO PARA DESPLIEGUE

Entregable 7.4 — Dictamen técnico para gerencia

Para: Gerencia de Operaciones, Cadena de Farmacias De: Equipo de Consultoría Técnica, Grupo 7 (Evaluación en Producción) Asunto: Causas del gap entre simulación y producción del agente de reabastecimiento

Resumen ejecutivo. El agente reporta una recompensa promedio de 51.2 en simulación, pero en producción genera 23 stockouts semanales, 41 unidades vencidas semanales y un costo de almacenamiento de 8,400 (2.6x el objetivo de 3,200). Con la información completa de los Grupos 1 a 6, incluida una ablación cuantitativa de 7 configuraciones distintas que verificamos nosotros mismos, el diagnóstico final es que *el reward está mal diseñado* (comprobado analíticamente) y el espacio de estados real es varias veces más grande de lo declarado. Eso hace que el algoritmo no converja de forma sana bajo ninguna combinación parcial de correcciones; solo las cuatro correcciones (MDP,

reward, algoritmo, exploración) aplicadas juntas producen una política razonablemente diversa.

Causas confirmadas (evidencia en código propio y de los Grupos 2, 3, 5 y 6):

1. Reward hacking, estructural y demostrado analíticamente. `inventory_reward` tiene rango [0,50]; la penalización combinada máxima es -12, así que ninguna combinación de penalizaciones compensa inventario alto. Confirmado independientemente por el Grupo 2 (sobre 220 estados) y el Grupo 6 (análisis de rango).
2. No-convergencia real, no "convergencia sana hacia óptimo local". La tabla de curvas del sistema original (TD 8.42 a 5.21, varianza 0.8, política 94% pedir_50) no es reproducible: tres grupos independientes (G3, G5, G6) midieron un espacio de estados real de 1,673 a 4,371 estados, contra 200 nominales. La reproducción de G6 (seed=123, verificada por nosotros ejecutando su notebook completo) da TD error de cola 51.33 (sube, no baja), varianza 127,917, y política dominante `pedir_0` en 80.2% de estados, el signo opuesto al 94% pedir_50 del enunciado.
3. Ninguna corrección parcial resuelve el problema, verificado con una ablación de 7 configuraciones (Baseline, G1, G2, G1+G2, G3, G5, las 4 juntas), corrida por G6 y completada por nosotros con la fila faltante G1+G2:

Configuración	TD error	Varianza	Política dominante	--- --- --- ---	Baseline
51.33	127,917	pedir_0 (80.2%)	Solo G1 (MDP)	8.42, mejor de la tabla	21,186
pedir_10 (48.9%)	Solo G2 (reward)	35.80	39,849	pedir_10 (25.5%, la más diversa)	G1+G2 combinados
24.16	45,968	pedir_10 (90.6%, la más concentrada después del baseline)	Solo G3 (algoritmo)	11.09-13.92	57,028-
94,284	pedir_0 (≈80%, ≈ baseline)	Solo G5 (exploración)	70.24-94.76, peor de la tabla	226,156-232,222, peor de la tabla	pedir_0 (≈48%)
Las 4 juntas	23.14	42,613	pedir_10 (74.5%)		

El hallazgo clave, verificado por nosotros ejecutando el notebook completo de G6: MDP+reward combinados, sin algoritmo ni exploración, generan su propio colapso de política (90.6% en `pedir_10`), peor que aplicar cualquiera de las dos correcciones por separado. Algoritmo (G3) y exploración (G5), pese a no ayudar cuando se aplican solos, son indispensables para deshacer ese colapso cuando se combinan con G1+G2: bajan la concentración de 90.6% a 74.5%.

4. Evaluación sin poder de detección. `evaluate_policy` mide el mismo entorno y reward que `train`; no puede exponer ninguno de los problemas anteriores antes del despliegue (Entregable 7.3).

Recomendación de despliegue, revisada, ya no es por fases. Dado que ninguna combinación parcial de correcciones produce una política estable y diversa,

recomendamos aplicar las cuatro correcciones (MDP, reward, algoritmo, exploración) juntas desde el primer despliegue, no de forma incremental. Un despliegue por fases que se detenga en "solo MDP+reward" dejaría el sistema en un colapso de política distinto al original (90.6% concentrado en una sola acción), no en un punto de mejora intermedio seguro.

Discrepancias metodológicas entre grupos, sin resolver, con transparencia hacia gerencia:

- G3 (entorno sin discretizar, 4,371 estados) y G5 (entorno discretizado, 200 estados) miden cobertura sobre definiciones de entorno distintas; sus porcentajes no son comparables directamente.
- El enunciado original afirma "pedir_50 en 94%"; las reproducciones de G3/G5/G6 apuntan más bien a un colapso hacia "pedir 0". No podemos afirmar cuál cifra es la correcta sin que los grupos reconcilien semillas y configuraciones.

Nota sobre trazabilidad: el notebook fuente de G6 (`S10 - Verificacion Empirica Reward Hacking.ipynb`) fue recibido completo, ejecutado por nosotros de punta a punta, y verificado: reproduce exacto los valores que G6 había reportado por texto (Baseline/G1/G2). La fila "G1+G2 combinados", que faltaba en su tabla original, la calculamos nosotros insertando el experimento en la posición exacta de su secuencia, para preservar el estado de `np.random` global del que depende la reproducibilidad.

Insumo recibido — Grupo 1 (MDP corregido)

Entregable 1.3/1.4 del Grupo 1: `mdp_corregido.py` (código completo en la carpeta del proyecto). Cambios sobre el MDP original:

- Estado pasa de 3 a 5 variables: agrega `unidades_en_transito` (pedido de ayer aún en camino, restaura la propiedad de Markov, Entregable 1.1) y `tendencia_demanda` (permite que la demanda cambie dentro del episodio en vez de quedar congelada, Entregable 1.2).
- `transition()` ahora es estocástica (`np.random.choice` sobre `PROB_DEMANDA / PROB_TENDENCIA`) y devuelve también `demanda_no_atendida` , la cantidad exacta de unidades en falta, dato que la transición original descartaba al recortar con `max(0, ...)` .
- Incluye `estado_a_original()` / `estado_desde_original()` para proyectar entre el formato de 3 y de 5 variables, y `transition_determinista()` para depurar sin ruido aleatorio.

Ejecutamos su `tabla_impacto()` para verificar el código y confirmar las cifras antes de citarlas en las preguntas de integración:

```
In [4]: import mdp_corregido as g1
```

```
impacto_g1 = g1.tabla_impacto()
```

Dimension	Original	Corregido
Niveles de inventario	10	10
Dias hasta vencimiento	5	5
Niveles de demanda	4	4
Unidades en transito (nuevo)	---	6
Tendencia de demanda (nuevo)	---	3
Total de estados	200	3600
Pares (estado, accion)	1200	21600
Entradas de la tabla Q	1200	21600

Factor de expansion: 18.0

Explicación matemática del tamaño del espacio

El tamaño de un espacio de estados discreto se obtiene multiplicando la cantidad de valores posibles de cada variable. Con los 10 niveles de inventario declarados, 5 niveles de vencimiento y 4 niveles de demanda, el modelo original usa

$$|S_{\text{original}}| = 10 \cdot 5 \cdot 4 = 200.$$

El MDP corregido añade 6 valores posibles de unidades en tránsito y 3 tendencias de demanda:

$$|S_{\text{corregido}}| = 10 \cdot 5 \cdot 4 \cdot 6 \cdot 3 = 3600.$$

Como existen 6 acciones, la cantidad de pares estado-acción es $|S \times A| = |S||A|$: $200 \cdot 6 = 1200$ en el modelo original y $3600 \cdot 6 = 21600$ en el corregido. El factor de expansión es

$$\frac{|S_{\text{corregido}}|}{|S_{\text{original}}|} = \frac{3600}{200} = 18.$$

Es decir, el modelo corregido tiene 18 veces más estados y requiere 18 veces más entradas en una tabla Q tabular. El cálculo sigue los 10 niveles declarados por el enunciado; la lista literal de 0 a 100 en pasos de 10 contiene 11 valores y produciría 220 estados originales.

Insumo recibido — Grupo 2 (recompensa corregida)

Entregable 2.1/2.3/2.4 del Grupo 2 (Grupo_2_Para_Integracion.ipynb):

- 2.1, reward hacking demostrado: sobre 220 estados, `R_original(pedir 50) - R_original(pedir 0)` es siempre positivo (mínimo 0.5). Confirma matemáticamente lo que ya inferimos en 7.1: pedir el máximo domina en todos los estados bajo el reward original.
- 2.3, reward corregido con 5 componentes: servicio (satura en 8, no sigue creciendo con más pedido), faltante (-2/unidad no atendida), exceso (-0.15/unidad sobre 2

días de demanda), vencimiento (riesgo gradual: $-0.45 * \max(0, 7 - \text{días}) / 7 * \text{inventario}$, ya no es el escalón binario del original), pedido (costo convexo $-0.04q - 0.002q^2$, penaliza más pedir 50 que pedir 10). Con esto la mejor acción ya varía por estado (ej. $(0, 1, 'crítico')$ pide 40; $(100, 14, 'crítico')$ pide 0), a diferencia del original donde 50 siempre gana.

- 2.4, simulación comparativa (1000 episodios x 30 pasos, semilla 3104): `pedir_50` da recompensa media -842.29 (inventario promedio 97.98, ventana de vencimiento 18.11/30 pasos, 1500 unidades pedidas); `política_corregida` da +144.40 (inventario promedio 5.76, ventana de vencimiento 7.48/30, 582.94 unidades pedidas). El reward corregido invierte el signo de la política dominante actual.
- Dato clave para Pregunta 2 (causa raíz): en su simulación, `stockout_promedio = 0` para ambas políticas, incluyendo `pedir_50` . Con la transición determinista tal cual está en `S10 - Codigo Examen.py` , ni el reward original ni el corregido generan stockouts por sí solos; la transición nunca dejaría inventario por debajo de la demanda si se pide lo suficiente. Es evidencia a favor de que los 23 stockouts de producción no se explican por el diseño del reward ni de la transición determinista, sino por algo que este experimento no captura: cobertura de exploración insuficiente (Q-learning tabular con estados nunca visitados, Entregable 7.2) y/o demanda real más variable que el `demand_map` fijo (Entregable 1.1/1.2 del Grupo 1).
- Handoff explícito de G2 para nosotros: usar los 41 vencimientos y el costo 2.63x como evidencia compatible con sobreabastecimiento, sin asignar los 23 stockouts únicamente a la recompensa.

Insumo recibido — Grupo 5 (exploración)

Entregable 5.3/5.4 del Grupo 5 (`parcial.py` , ejecutado y verificado en esta sesión). Su entorno (`PharmacyInventoryEnv`) discretiza el inventario a la grilla de 10 niveles, a diferencia del entorno que replicó G3, que no discretiza: 200 estados nominales, 1200 pares (s, a) . Comparan `epsilon-greedy` ($\epsilon=0.05$, el original) contra `Optimista + UCB` (valor optimista 100 + bono UCB), ambos con 1000 episodios:

Métrica	ϵ -greedy (original)	Optimista+UCB (propuesto)
Pares (s,a) visitados	770 / 1200	1195 / 1200
% cobertura	64.2%	99.6%
Estados con 0 visitas	0 / 200	0 / 200
Pares no visitados en demanda crítico	150	5
Estados con stockout directo (política greedy pide menos de lo que hace falta)	6	-
De esos, con 3 o menos de 6 acciones probadas ("pobre exploración")	5	-

Ejecutado por nosotros para verificar: corre limpio, resultados reproducibles con semilla 42. Los 6 estados con stockout directo son todos de inventario bajo (10-30) y demanda alta/crítica, coherente con nuestra hipótesis de 7.2, pero la cifra real (6 estados, 5 por poca exploración) es mucho menor que la que habíamos estimado con la lógica del 94% (~12/200) porque este entorno (200 estados discretizados) es distinto del que usa G3 para su cifra de 4,371.

```
In [5]: import subprocess

resultado_g5 = subprocess.run(
    ['python3', 'exploracion_grupo5.py'],
    capture_output=True, text=True, cwd='.'
)
print(resultado_g5.stdout)
```

Espacio: 200 estados x 6 acciones = 1200 pares (s,a)

Entrenando epsilon-greedy (epsilon=0.05)

Entrenando Optimista + UCB

METRICA	eps-greedy	Opt+UCB
Pares (s,a) visitados	770	1195
% cobertura	64.2%	99.6%
Pares NO visitados	430	5

Pares NO visitados por nivel de demanda:

bajo	eps-greedy:	90	Opt+UCB:	0
medio	eps-greedy:	82	Opt+UCB:	0
alto	eps-greedy:	108	Opt+UCB:	0
crítico	eps-greedy:	150	Opt+UCB:	5

Pares NO visitados por accion:

pedir 0	eps-greedy:	4	Opt+UCB:	0
pedir 10	eps-greedy:	78	Opt+UCB:	0
pedir 20	eps-greedy:	88	Opt+UCB:	0
pedir 30	eps-greedy:	82	Opt+UCB:	1
pedir 40	eps-greedy:	89	Opt+UCB:	2
pedir 50	eps-greedy:	89	Opt+UCB:	2

DISTRIBUCION DE VISITAS POR PAR (s,a)

eps-greedy:

Pares con 0 visitas:	430
Pares con 1-5 visitas:	389
Pares con 6-20 visitas:	170
Pares con 21-100 visitas:	71
Pares con >100 visitas:	140

Promedio (pares visitados):	77.9
Desviacion estandar:	208.3
Minimo:	1
Maximo:	2407

Optimista+UCB:

Pares con 0 visitas:	5
Pares con 1-5 visitas:	1025
Pares con 6-20 visitas:	106
Pares con 21-100 visitas:	39
Pares con >100 visitas:	25

Promedio (pares visitados):	50.2
Desviacion estandar:	338.6
Minimo:	1
Maximo:	3608

DISTRIBUCION DE VISITAS POR ESTADO


```

=====

eps-greedy:
  Estados con 0 visitas:          0
  Estados con 1-50 visitas:      96
  Estados con 51-200 visitas:    26
  Estados con 201-500:          36
  Estados con >500 visitas:      42
  ---
  Promedio:      300.0
  Desv. std:     461.0

Optimista+UCB:
  Estados con 0 visitas:          0
  Estados con 1-50 visitas:      165
  Estados con 51-200 visitas:    10
  Estados con 201-500:          0
  Estados con >500 visitas:      25
  ---
  Promedio:      300.0
  Desv. std:     783.1

Estados donde la politica causa stockout directo:
      Estado                      Accion  Demanda  Acciones probad
as
-----
-
(10, 14, alto    ) pedir 0 necesita 25      1/6 acciones
(10, 30, medio   ) pedir 0 necesita 15      1/6 acciones
(10, 30, alto    ) pedir 0 necesita 25      2/6 acciones
(30, 14, crítico ) pedir 0 necesita 40      1/6 acciones
(30, 30, crítico ) pedir 0 necesita 40      1/6 acciones

Total estados con stockout directo: 6
De estos, con pobre exploracion (<=3 acciones probadas): 5

```

Insumo recibido — Grupo 6 (convergencia)

Entregable 6.1/6.4 del Grupo 6, texto y tabla (fuente: `S10 - Verificacion Empirica Reward Hacking.ipynb`, seed=123, 1000 episodios; archivo no adjunto a nuestro pull originalmente, no lo ejecutamos ni verificamos nosotros mismos en esta primera pasada; se cita textual como lo recibimos).

6.1, diagnóstico: ambos (reward hacking + no-convergencia), no "convergencia sana hacia óptimo local".

- Reward hacking, estructural (analítico): `inventory_reward` tiene rango [0,50]; la penalización combinada máxima (`expiry_penalty + order_penalty`) es -12. Ninguna combinación de penalizaciones compensa inventario alto, así que el óptimo de esa función es pedir el máximo siempre, sin importar demanda ni vencimiento. Es un diseño roto, no un efecto secundario menor.

- No-convergencia real (empírico, contradice la narrativa original): su reproducción del baseline dio TD error de cola 51.33 (sube, no baja), varianza de reward 127,917 (no el 0.8 que reporta el sistema original), y política dominante real pedir_0 en 80.2% de estados (no pedir_50 en 94% como afirma el enunciado). Causa técnica: `transition()` no respeta la grilla de 200 estados (demanda 5/15/25 no es múltiplo de las acciones de a 10), así que el estado real visitado explota a 1,673 estados en su corrida, contra 200 nominales.
- Cruce explícito con G3: su hallazgo (TD original 33.8 a 35.3, no baja de forma consistente) coincide con esta reproducción. Recomendamos retirar la lectura "convergencia sana" basada en la tabla 8.42 a 5.21 del sistema original, ya que no es reproducible con el código tal cual está.

6.4, proyección/ablación con las 4 correcciones ya recibidas (mismo entorno y semilla):

Experimento	Reward	TD error	Varianza	Acción dominante
Baseline original	289.62	51.33	127,917	pedir_0 (80.2%)
Solo G1 (MDP)	-10.70	8.42, mejor	21,185	pedir_10 (48.9%)
Solo G2 (reward)	-627.73	35.80	39,849	pedir_10 (25.5%, más repartida)
Solo G3 (algoritmo)	93.03	11.09	57,028	pedir_0 (80.7%, ≈ baseline)
Solo G5 (exploración)	590.70	94.76, peor	226,156, peor	pedir_0 (47.8%)
Las 4 combinadas	-636.60	23.14	42,612	pedir_10 (74.5%)

Lectura de G6 por corrección: G1 solo rompe el colapso en `pedir_0` y da el TD más limpio de la tabla, ataca la causa raíz (explosión de estados). G2 solo es el que más diversifica la política por estado (25.5%, mínimo de la tabla), mejor candidato teórico contra stockout, pero sin G1 no converge limpio (TD 35.8). G3 solo casi no mueve nada frente al baseline. G5 solo empeora todo (peor TD, peor varianza) en su pipeline combinado. Las 4 juntas bajan el TD (23.14) pero la política vuelve a concentrarse (74.5% en `pedir_10`), mejor que el baseline pero menos adaptativa que G2 solo.

Respuesta directa de G6 a nuestra hipótesis de Pregunta 1: parcial, con matiz. MDP (G1) solo ya ataca stockouts, no solo costo: rompe el `pedir_0` que causa los 23 stockouts/semana. Reward (G2) solo es el que más diferencia política por demanda, pero desestabiliza el TD sin G1. Algoritmo (G3) y exploración (G5) solos no aportan nada por sí mismos en su pipeline; sirven para estabilizar el paquete combinado, no lo sustituyen. Advertencia de G6: combinar las 4 no da lo mejor de cada una. La política combinada (74.5% concentrada) retrocede frente a G2 solo (25.5%); el plan mínimo viable con las 4 correcciones puede seguir sin resolver bien los stockouts en demanda alta/crítica si `pedir_10` no escala con el nivel de demanda.

Insumos pendientes de otros grupos

Tabla llenada durante la sesión presencial. Se conservan las preguntas como guía de qué se le pidió a cada grupo.

Grupo	Qué necesitábamos	Pregunta concreta que se hizo	Estado
1 (MDP)	Entregable 1.3 (MDP corregido) y 1.4 (tabla de tamaño de espacio)	¿Cuántos estados tiene el MDP corregido? ¿La nueva transición de demanda hace que los estados críticos se visiten más seguido de forma natural, o el problema de cobertura persiste igual?	Recibido (<code>mdp_corregido.py</code>)
2 (Recompensa)	Entregable 2.3 (función corregida, 3+ componentes) y 2.4 (tabla comparativa de recompensa acumulada)	¿Cuál es la magnitud de cada componente? ¿Cuánto reduce la recompensa acumulada de "pedir siempre 50" frente a una política razonable?	Recibido (<code>Grupo_2_Para_Integracion.ipynb</code>)
3 (Algoritmo)	Entregable 3.3 (train corregido) y 3.4 (curvas original vs. corregido)	¿Cuál era el error exacto en <code>train</code> ? ¿Qué algoritmo implementaba realmente el código original? ¿Cómo cambia el error TD y la varianza con la corrección?	Recibido (respuesta directa + <code>resumen_ablacion.json</code> , código no adjunto)
4 (Aproximación)	Entregable 4.3 (preprocess_state mejorado) y 4.4 (tabla de MSE)	¿Qué normalización usaron? ¿Sus características	Recibido (respuesta directa, <code>preprocesamiento_mejorado.py</code> no adjunto)

Grupo	Qué necesitábamos	Pregunta concreta que se hizo	Estado
		nuevas dependen de la representación de estado actual o de la corregida por el Grupo 1?	
5 (Exploración)	Entregable 5.3 (métricas de cobertura) y 5.4 (stockouts atribuibles a no-cobertura)	¿Cuántos estados y pares (s,a) de los 200/1200 se visitaron realmente en 1000 episodios con $\epsilon=0.05$? ¿Cuántos de los 23 stockouts atribuyen a estados no visitados?	Recibido (<code>parcial.py</code> , ejecutado y verificado)
6 (Convergencia)	Entregable 6.1 (diagnóstico: óptimo local / reward hacking) y 6.4 (proyección de curvas post-corrección)	¿Su diagnóstico es reward hacking, óptimo local, o ambos? ¿Cómo proyectan que cambie el error TD y la política dominante si se aplican las correcciones de los Grupos 1, 2 y 3 juntas?	Recibido completo (<code>S10 - Verificacion Empirica Reward Hacking.ipynb</code> , ejecutado y verificado; incluye la fila "G1+G2 combinados" que faltaba, calculada por nosotros preservando su secuencia de semilla)

Regla de registro: al recibir un insumo, se pega textual en la celda de la Pregunta de integración correspondiente, sin resumir de memoria, y se actualiza el estado aquí.

Hallazgo transversal de G3, aplica a nuestro propio Entregable 7.2: su réplica encontró 4,371 estados alcanzables en el entorno original, no los 200 (ni los 220 literales de la grilla) que usamos como denominador en 7.2. Es consistente con lo que ya habíamos notado en 7.1: `transition()` no discretiza el inventario a la grilla declarada. Nuestra estimación de "~12/200 estados no visitados" en 7.2 probablemente subestima el problema de cobertura real; ver la nota agregada al final de 7.2 y la respuesta actualizada de Pregunta 2.

Contradicción metodológica G3 vs. G5, sin resolver: el entorno de G3 replica `transition()` del `.py` original sin discretizar el inventario (de ahí sus 4,371 estados alcanzables). El entorno de G5 (`parcial.py`) sí discretiza el inventario con `_discretize_inventory()` a la grilla de 10 niveles; su `PharmacyInventoryEnv` tiene exactamente 200 estados nominales y ningún estado con 0 visitas tras 1000 episodios. Son dos ambientes distintos probando la misma pregunta de cobertura; sus porcentajes de cobertura no son comparables directamente entre sí (5.2% de G3 sobre 26,226 pares vs. 64.2% de G5 sobre 1,200 pares).

Tercer dato de tamaño de estado, G6 (reconcilia, no contradice): su reproducción del baseline (seed=123, 1000 episodios) visitó 1,673 estados de forma empírica en una sola corrida, un subconjunto razonable de los 4,371 alcanzables que G3 midió por enumeración, no una tercera cifra contradictoria. Confirma otra vez, por una cuarta vía independiente, que `transition()` no respeta la grilla de 200 estados declarada.

Nueva discrepancia, G5 vs. G6 sobre el efecto de la exploración sola: G5 reportó que su estrategia Optimista+UCB, aislada, sube la cobertura de 64.2% a 99.6% (mejora clara). G6, en su tabla de ablación "Solo G5" dentro de su propio pipeline combinado, reporta que la exploración sola empeora TD error y varianza. No es necesariamente una contradicción, probablemente miden cosas distintas (cobertura vs. TD/varianza de reward) sobre setups distintos.

Cierre del pendiente de G6: recibimos el notebook completo de G6 (`S10 – Verificacion Empirica Reward Hacking.ipynb`), lo ejecutamos y confirmamos que reproduce exacto sus valores publicados (Baseline/G1/G2). Insertamos la fila "G1+G2 combinados" que faltaba, en la posición exacta de su secuencia de experimentos (justo después de "Solo G2", ya que el entrenamiento consume `np.random` global y el orden de ejecución afecta el resultado). Resultado: reward -646.64, TD 24.16, varianza 45,968.30, política dominante `pedir_10` en 90.6% de estados. Resuelve la ambigüedad de Pregunta 1 (ver ahí el detalle completo).

Preguntas de integración

Pregunta 1 — Diagnóstico sistémico

Insumos usados: MDP corregido (Grupo 1), función de recompensa corregida (Grupo 2), proyección de curvas (Grupo 6), completada con la fila "G1+G2 combinados" (ver abajo).

Fila faltante, calculada por nosotros: no teníamos el archivo `S10 – Verificacion Empirica Reward Hacking.ipynb` de G6 cuando pedimos la fila "G1+G2 combinados"; después lo recibimos completo, con todas las funciones (`transition_g1_fix` , `reward_g2_fix` , `run_experiment` , semilla=123). En vez de reenviar el pedido, corrimos nosotros mismos esa fila insertando

`run_experiment("G1+G2 combinados", transition_g1_fix, reward_g2_fix, alpha_original, epsilon_original)` en la posición exacta que preserva la secuencia de números aleatorios de su notebook, justo después de "Solo G2" y antes de "Solo G3" (el orden importa porque el entrenamiento usa `np.random` global, no una semilla por experimento). Verificamos que Baseline/G1/G2 reprodujeron exacto los valores publicados por G6 (289.62/-10.70/-627.73 de reward, 51.33/8.42/35.80 de TD) antes de confiar en la fila nueva:

Experimento	Reward	TD error	Varianza	Acción dominante
Baseline (original)	289.62	51.33	127,917.44	pedir_0 (80.2%)
Solo G1 (MDP)	-10.70	8.42	21,185.62	pedir_10 (48.9%)
Solo G2 (reward)	-627.73	35.80	39,849.11	pedir_10 (25.5%)
G1+G2 combinados	-646.64	24.16	45,968.30	pedir_10 (90.6%)
Las 4 combinadas (G1+G2+G3+G5)	-636.60	23.14	42,612.74	pedir_10 (74.5%)

Esto resuelve la ambigüedad que habíamos dejado pendiente, y la respuesta es que MDP+reward combinados ya se recolapsan por su cuenta, sin necesidad de que G3/G5 lo arruinen. La política se concentra en `pedir_10` en 90.6% de estados, más concentrada que las 4 correcciones juntas (74.5%) y muchísimo más que G2 solo (25.5%, el mínimo de toda la tabla). G3 y G5, lejos de ser prescindibles, sí aportan algo real: diversifican la política de 90.6% de vuelta a 74.5% cuando se agregan sobre G1+G2. La causa del colapso en G1+G2 es coherente con lo que ya sabíamos:

`epsilon=0.05` fijo sigue sin cambiar entre G1+G2, así que la exploración sigue siendo insuficiente para el espacio de estados ampliado que produce el MDP estocástico de G1 (ver Pregunta 2). El TD error de G1+G2 (24.16) es peor que G1 solo (8.42), señal de que agregar el reward de G2 sin ajustar exploración desestabiliza lo que G1 solo ya había logrado.

Respuesta final a la pregunta original: si se aplican únicamente las correcciones del MDP y del reward, sin cambiar algoritmo ni exploración, el agente sí aprende algo mejor que el baseline (rompe el colapso original hacia `pedir_0`, y el TD baja de 51.33 a 24.16), pero la política resultante es más concentrada y menos adaptativa que cualquiera de las dos correcciones por separado. MDP+reward combinados generan su propio colapso hacia una acción distinta (`pedir_10` en vez de `pedir_0`), no una política verdaderamente sensible al estado. Algoritmo (G3) y exploración (G5) sí son necesarios para acercarse a la diversidad de política que G2 alcanza por sí solo, así que el plan mínimo viable (Pregunta 3) no puede quedarse en solo G1+G2.

Pregunta 2 — Causa raíz

Insumos usados: algoritmo corregido (Grupo 3), métricas de cobertura (Grupo 5), análisis de gap (Grupo 7, propio: Entregables 7.1/7.2).

Grupo 2 (referencia cruzada): en su simulación con política óptima computada directamente (sin aprendizaje tabular), `stockout_promedio = 0` bajo cualquier política, lo que apuntaba a que la causa está en el proceso de aprendizaje, no en el reward ni en la transición.

Grupo 3, el bug real (no es SARSA accidental): la fórmula de actualización es Q-learning off-policy correcto matemáticamente. El bug real es que `np.argmax` no desempata al azar: con `Q` inicializado en ceros, todo estado no visitado empata en sus 6 acciones y la política greedy elige siempre el índice más bajo, la acción 0 (no pedir). Sobre su réplica del entorno sin discretizar (4,371 estados alcanzables), la cobertura final de pares `(s,a)` es solo 5.23% de 26,226.

Grupo 5, cobertura medida directamente sobre el entorno discretizado (200 estados, 1200 pares): ϵ -greedy ($\epsilon=0.05$) cubre 64.2% de los pares tras 1000 episodios; el déficit se concentra en demanda `crítico` (150/1200 pares sin visitar). Esto produce 6 estados con stockout directo bajo la política greedy, 5 de los cuales tienen 3 o menos de las 6 acciones probadas.

Grupo 6, confirma con una cuarta medición independiente que el espacio real explota más allá de la grilla declarada: su reproducción del baseline visitó 1,673 estados en una sola corrida (subconjunto razonable de los 4,371 alcanzables de G3), y encontró que la política dominante real del sistema original es `pedir_0` en 80.2% de estados, no `pedir_50` en 94% como afirma el enunciado del examen. Es una corrección directa a un dato que habíamos tomado como dado desde 7.1/7.2: la política dominante reportada en el sistema original no es reproducible tal cual, y en la reproducción de G6 va en la dirección opuesta (colapso hacia "no pedir", no hacia "pedir el máximo").

Contradicción/matiz a resolver, sin esconderla: el enunciado original afirma "pedir_50 en 94% de estados"; nuestro propio análisis (7.1) tomó ese dato como evidencia de reward hacking hacia sobreabastecimiento. La reproducción de G6 (`pedir_0` en 80.2%) y de G3/G5 (colapso hacia baja cobertura, default a acción de índice más bajo) apuntan más bien a que el comportamiento real y reproducible del sistema es subabastecimiento por defecto, no sobreabastecimiento. Ambos pueden ser ciertos en simultáneo si el 94%/pedir_50 del enunciado viene de una corrida o semilla distinta a las que replicaron G3/G5/G6, pero no podemos afirmar cuál es la cifra correcta sin que los grupos reconcilien sus semillas y configuraciones. Para el dictamen (7.4), tratamos ambos síntomas (sobreabastecimiento y posible colapso a "no pedir") como plausibles y dependientes de la corrida específica, no como una cifra única y consistente.

Conclusión de causa raíz, reforzada con la fila G1+G2: el problema no es el reward ni el MDP en sí (confirmado independientemente por G2 y por la ausencia de stockouts bajo política óptima), sino la combinación de (a) el bug de desempate en `argmax`, que sesga sistemáticamente hacia "no pedir" en estados poco visitados, (b) una tasa de exploración fija (`epsilon=0.05`) que no compensa esa falta de cobertura en demanda

crítica y que, sin cambios, sigue produciendo colapso de política incluso después de corregir MDP y reward juntos (90.6% en pedir_10), y (c) un espacio de estados real varias veces más grande que el declarado (200 nominal vs. 1,673-4,371 medidos por tres grupos distintos de forma independiente). La exploración (G5) y el algoritmo (G3) no son un extra opcional: son los que devuelven la diversidad de política de 90.6% a 74.5% al combinarse con G1+G2.

Pregunta 3 — Plan de corrección mínimo viable

Insumos usados: resultados de todos los grupos, incluida la ablación cuantitativa completa de G6 con la fila G1+G2 ya cerrada.

Prioridad	Componente que modifica	Grupo fuente	Métrica que mejora	Mejora esperada (medida)	Justificación
1	MDP (restaurar Markov, no explotar espacio de estados sin control)	Grupo 1	TD error y colapso de política hacia pedir_0	TD 51.33 a 8.42 (el mejor de toda la tabla); política dominante 80.2% a 48.9%	Corrección individual de mayor impacto, ataca la causa raíz de por qué el agente converge mal
2	Recompensa	Grupo 2	Diversidad de política por nivel de demanda; costo de almacenamiento	Política dominante cae a 25.5% (la más repartida de toda la tabla, mejor que G1+G2 combinados con 90.6%); inventario promedio 97.98 a 5.76 unidades (G2)	Es la corrección que más alinea la acción con el estado real, pero no debe aplicarse sola con G1: la combinación G1+G2 sin más recolapsa la política (90.6% en pedir_10)
3	Algoritmo + exploración (desempate aleatorio, decaimiento α/ϵ , Optimista+UCB)	Grupos 3 y 5	Recuperar diversidad de política que G1+G2 pierde por su cuenta; cobertura en demanda crítica	Baja el colapso de 90.6% (G1+G2) a 74.5% (las 4 juntas); TD de 24.16 a 23.14	Ya no es "estabilización opcional": la fila G1+G2 confirma que sin G3/G5, MDP+reward solos generan su propio colapso de política

Conclusión revisada, con la fila G1+G2 ya cerrada: las 4 correcciones son necesarias para el plan mínimo viable. Ninguna combinación parcial (G1 solo, G2 solo, ni G1+G2)

alcanza la diversidad de política de G2 aislado (25.5%), y G1+G2 sin G3/G5 es, contraintuitivamente, la configuración más concentrada de toda la tabla después del baseline. Recomendación final: aplicar las 4 correcciones juntas desde el inicio, no por fases. La fase intermedia "solo G1+G2" no es un punto de mejora estable, es un colapso distinto al original. Ejecutable en dos semanas: las 4 correcciones ya están escritas y probadas por sus respectivos grupos.

Pregunta 4 — Compatibilidad de correcciones

Insumos usados: MDP corregido (Grupo 1), preprocesamiento mejorado (Grupo 4), algoritmo corregido (Grupo 3).

Grupo 1, riesgo de incompatibilidad confirmado en el código: `mdp_corregido.py` cambia el estado de 3 a 5 variables. El `preprocess_state` original y el `LinearApproximator(n_features=3, ...)` del Grupo 4 están escritos para 3 variables.

Grupo 4, respuesta directa: trabajan enteramente sobre el estado original de 3 variables, expandido a 12 features (normalización correcta, demanda como one-hot). No usan `unidades_en_transito` ni `tendencia_demanda`. Confirman que si se adopta el MDP de 5 variables de G1, necesitan que el estado llegue ya reducido a `(inventory, days_to_expiry, demand_level)`.

Grupo 3, respuesta directa: mecánicamente, `Q = defaultdict(lambda: np.zeros(6))` no tiene problema con tuplas de 5 elementos en vez de 3. El problema real es de calibración: su esquema de decaimiento de `alpha / epsilon` ya está calibrado contra su propio hallazgo de 4,371 estados alcanzables del MDP original. Si el espacio crece más con el MDP de G1, la misma calibración cubre una fracción todavía menor con el mismo presupuesto de episodios.

Conclusión de compatibilidad, las tres correcciones son técnicamente compatibles pero no sinérgicas sin coordinación adicional:

1. G1 y G4 son compatibles vía `estado_a_original()` (adaptador que ya escribí G1), pero G4 pierde la señal de las 2 variables nuevas. No hay error, hay pérdida de información.
2. G1 y G3 son compatibles en el mecanismo, pero incompatibles en calibración: aplicar el MDP de G1 sin que G3 recalibre `alpha / epsilon` contra el nuevo tamaño de espacio empeora la cobertura, confirmado con dato duro en Pregunta 1: G1+G2 sin G3/G5 colapsa la política a 90.6% en una sola acción.
3. G3 y G5 corrigen exploración/algoritmo pero sobre definiciones distintas del entorno (G3 sin discretizar, G5 discretizado a 200 estados); antes de combinar sus dos correcciones en un solo pipeline, hay que decidir sobre cuál de los dos entornos se entrena en producción.

4. Orden recomendado, revisado, ya no es "por fases": dado que G1+G2 sin G3/G5 recae la política (Pregunta 1), las cuatro correcciones deben integrarse juntas desde el inicio, no de forma incremental. La compatibilidad técnica ya está resuelta (adaptador de G1, mecanismo de G3); lo único pendiente es que G3 recalibre su esquema de decaimiento contra el tamaño de espacio real que resulta de aplicar el MDP de G1.

Reflexión grupal

Las hipótesis planteadas en 7.1/7.2 resultaron parcialmente correctas, pero incompletas. Confirmamos que el MDP original no era representativo del entorno real y que la función de recompensa estaba mal definida: G1 corrigió el MDP agregando `unidades_en_transito` y `tendencia_demanda`, y evitando que la transición trunque el inventario a cero; G2 corrigió el reward con 5 componentes que penalizan exceso y falta de inventario, resolviendo el reward hacking observado inicialmente. Sin embargo, nuestra propia ablación (fila "G1+G2 combinados") mostró que estas dos correcciones por sí solas no bastan: en vez de producir una política sensible al estado, G1+G2 sin más recae hacia una acción única (`pedir_10` en 90.6% de los estados), una concentración incluso mayor que la de cualquier corrección individual. El TD error también empeora respecto a G1 solo (24.16 contra 8.42), señal de que corregir MDP y reward juntos, sin tocar exploración, desestabiliza lo que G1 ya había logrado.

La causa raíz real, según el diagnóstico integrado con G3 y G5, no es el MDP ni el reward en sí (confirmado independientemente por G2, cuya política óptima no produce stockouts bajo ningún reward), sino la combinación de: (a) el bug de desempate en `np.argmax`, que sesga sistemáticamente hacia "no pedir" en estados poco visitados; (b) una tasa de exploración fija ($\epsilon = 0.05$) que no compensa la falta de cobertura en demanda crítica, incluso después de ampliar el espacio de estados con el MDP de G1; y (c) un espacio de estados real 8 a 20 veces mayor que el declarado (200 nominal contra 1,673-4,371 medidos independientemente por tres grupos). Quedó también una contradicción que no logramos resolver del todo: el enunciado original reportaba `pedir_50` en 94% de los estados (sobreabastecimiento), mientras que la reproducción de G6 encontró `pedir_0` en 80.2% (subabastecimiento). Ambos síntomas son plausibles según la semilla o configuración usada, pero no pudimos determinar cuál es la cifra correcta.

Por eso revisamos nuestra conclusión original: el plan mínimo viable no puede quedarse en MDP+reward. Las cuatro correcciones (MDP, reward, algoritmo, exploración) son necesarias y deben aplicarse juntas desde el inicio, no por fases. La fase intermedia "solo G1+G2" no es un punto de mejora estable, es un colapso distinto al del baseline.